

Final Project Specification: SE 3231 – Deep Learning Application

1. Project Goal

The goal of this project is to design, train, and deploy a Deep Learning model to solve a real-world problem. Students must demonstrate the ability to handle data preprocessing, architecture design, and model evaluation using **TensorFlow/Keras** (or PyTorch), integrated into a functional **User Interface (UI)**.

2. The "No Pre-Built AI" Constraint

To ensure students understand the mechanics of Deep Learning, the following rules apply:

- **Allowed:** TensorFlow, Keras, PyTorch, Scikit-learn, OpenCV, NumPy.
- **Prohibited:** High-level "wrapper" libraries or pre-built services. You **cannot** use OpenAI APIs (GPT), Anthropic, LangChain, or pre-trained "black-box" chatbot frameworks.
- **Requirement:** You must define the layers, loss functions, and optimizers within your code.

3. Technical Requirements

A. Modeling Approaches

Students must choose one of the following two paths:

1. **Training from Scratch:** Design a custom architecture (e.g., CNN, RNN, or Dense Neural Network) and train it on a raw dataset.
2. **Transfer Learning:** Utilize a standard backbone (e.g., ResNet50, MobileNet, VGG16) as a feature extractor, then add and train custom classification/regression heads specifically for your unique dataset.

B. The User Interface (UI)

The model cannot live in a Jupyter Notebook alone. It must be wrapped in a UI that allows a non-technical user to:

- **Input Data:** Upload an image, record audio, or type text.
- **View Inference:** See the model's output in real-time.
- **Interpret Results:** Display confidence scores or class probabilities.
- *Recommended Tools:* Streamlit, Gradio, Flask, or FastAPI with a simple HTML frontend.

4. Sample Project Ideas

Project Category	Technology Focus	Description
Medical Imaging	CNN / Transfer Learning	A UI where users upload X-rays or skin scans; the model (using a ResNet backbone) detects anomalies like pneumonia or lesions.

Project Category	Technology Focus	Description
Real-time Sign Language	Computer Vision / Keras	Uses a webcam feed to recognize American Sign Language (ASL) gestures and displays the translated text on screen.
Audio Emotion Analysis	1D-CNN or RNN	Users record a 5-second audio clip; the model analyzes the frequency patterns to determine the speaker's emotional state.
Document Digitizer	OCR / Custom CNN	An app that takes a photo of handwritten notes and converts them into digital text using a custom-trained character recognition model.
Fraud Detection Portal	Deep Dense Networks	A dashboard for financial analysts to input transaction parameters and receive a "Risk Score" generated by a deep MLP.

5. Evaluation Criteria

- **Data Pipeline (20%):** How well was the data cleaned, normalized, and augmented?
- **Architecture Design (30%):** Selection of layers (Dropout, Batch Normalization, etc.) and justification for the chosen loss function and optimizer.
- **Model Performance (20%):** Analysis of training/validation curves, confusion matrices, and avoidance of overfitting.
- **UI Functionality (20%):** Is the interface intuitive and does it handle errors (e.g., wrong file types) gracefully?
- **Documentation (10%):** A clear README and code comments.

6. Final Deliverables

1. **Source Code:** Comprehensive Python scripts including the training logic and the UI deployment.
2. **Saved Model:** The `.h5` or `.keras` file representing the trained weights.
3. **README:** Describes how to use the app (or train a new model whatsoever)

7. Final Notes:

1. Deadline: May 4, 2026 11:59pm
2. Submission Link: To be posted in the leading days before the deadline.